Phonebook directory

MODULES

1.add contact

2.search contact

3.update contact

4.delete contact

5.display contact

6.statistics and file handling

REQUIREMENTS

1)HARDWARE : COMPUTER,4 GB RAM

2)SOFTWARE :C++ COMPILER

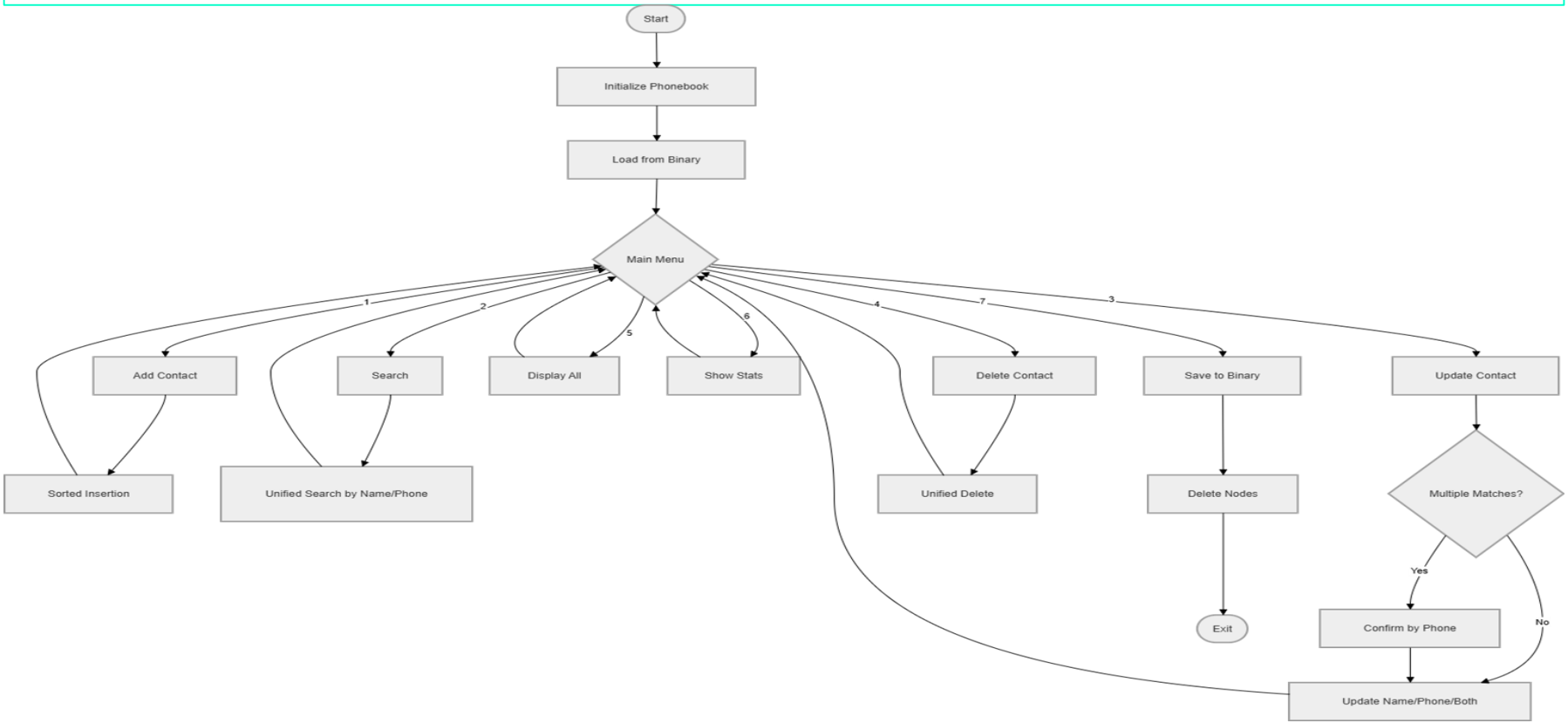
METHODOLOGY

1. Add Contact (Sorted Insertion)
2. Search Contact (Unified Search by Name/Phone)
3. Update Contact
4. Delete Contact
5. Display Contacts
6. Statistics & File Handling (Save/Load Binary Data)

ALGORITHM

1. START
2. LOAD DATA
3. DISPLAY MENU
4. PERFORM OPERATION
5. SAVE DATA
6. EXIT

FLOW CHART:


Start
Initialize Phonebook
Load from Binary
Main Menu
1
2
5
6
4
7
3
Add Contact
Search
Display All
Show Stats
Delete Contact
Save to Binary
Update Contact
Sorted Insertion
Unified Search by Name/Phone
Unified Delete
Delete Nodes
Multiple Matches?
Yes
No
Exit
Confirm by Phone
Update Name/Phone/Both

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <cstring>
#include <iomanip>

using namespace std;

#define nullptr NULL // Compatibility for older compilers

struct ContactData {
    char name[50];
    char phone[15];
};

struct Node {
    ContactData data;
    Node* next;
    Node* prev;
};

class Phonebook {
private:
    Node* head;

public:
    Phonebook() {
        head = NULL;
        loadFromBinary();
    }

    ~Phonebook() {
        saveToBinary();
        Node* curr = head;
        while (curr) {
            Node* next = curr->next;
            delete curr;
            curr = next;
        }
    }

    // Module 1: Sorted Insertion (Maintains A-Z)
    void addContact(string n, string p) {
        Node* newNode = new Node;
        strncpy(newNode->data.name, n.c_str(), 49);
        strncpy(newNode->data.phone, p.c_str(), 14);
        newNode->next = NULL; newNode->prev = NULL;

        if (!head || strcasecmp(newNode->data.name, head->data.name) < 0) {
            newNode->next = head;
            if (head) head->prev = newNode;
            head = newNode;
            return;
        }

```

```
    }

    Node* curr = head;
    while (curr->next && strcasecmp(newNode->data.name, curr->next->data.name
        ) > 0)
        curr = curr->next;

    newNode->next = curr->next;
    newNode->prev = curr;
    if (curr->next) curr->next->prev = newNode;
    curr->next = newNode;
}

// Module 2: Unified Search
void unifiedSearch(string query) {
    Node* curr = head;
    bool found = false;
    cout << "\n--- Search Results ---" << endl;
    while (curr) {
        if (strcasecmp(curr->data.name, query.c_str()) == 0 || strcmp(curr
            ->data.phone, query.c_str()) == 0) {
            cout << "Match: " << curr->data.name << " | " << curr->data.phone
                << endl;
            found = true;
        }
        curr = curr->next;
    }
    if (!found) cout << "No records found matching: " << query << endl;
```

```
// Module 3: Unified Delete
void unifiedDelete(string query) {
    Node* curr = head;
    bool deleted = false;
    while (curr) {
        if (strcasecmp(curr->data.name, query.c_str()) == 0 || strcmp(curr
            ->data.phone, query.c_str()) == 0) {
            if (curr->prev) curr->prev->next = curr->next;
            if (curr->next) curr->next->prev = curr->prev;
            if (curr == head) head = curr->next;

            Node* toDelete = curr;
            curr = curr->next;
            delete toDelete;
            deleted = true;
            continue;
        }
        curr = curr->next;
    }
    if (deleted) cout << "Record(s) deleted.\n";
    else cout << "Record not found.\n";
}

// Module 4: Integrated Update (Name, Number, or Both)
void updateContact(string query) {
    Node* curr = head;
    Node* matches[10];
    int matchCount = 0;
```

```
108
109         while (curr && matchCount < 10) {
110             if (strcasecmp(curr->data.name, query.c_str()) == 0 || strcmp(curr
                    ->data.phone, query.c_str()) == 0) {
111                 matches[matchCount++] = curr;
112             }
113             curr = curr->next;
114         }
115
116         if (matchCount == 0) {
117             cout << "Contact not found.\n";
118             return;
119         }
120
121         Node* target = (matchCount > 1) ? NULL : matches[0];
122         if (matchCount > 1) {
123             cout << "\nMultiple matches. Type the exact Phone Number to confirm:
                    ";
124             string confirm; getline(cin, confirm);
125             for (int i = 0; i < matchCount; i++) {
126                 if (strcmp(matches[i]->data.phone, confirm.c_str()) == 0) target
                        = matches[i];
127             }
128         }
129
130         if (!target) { cout << "Invalid Selection.\n"; return; }
131
132         cout << "\nUpdating: " << target->data.name << " | " << target->data
```

```
133         cout << "\n1. Update Name\n2. Update Number\n3. Update Both\nChoice: ";
134         int upChoice; cin >> upChoice; cin.ignore();
135
136         string oldPhone = target->data.phone;
137         string oldName = target->data.name;
138
139         if (upChoice == 1) { // Change Name only
140             string newName; cout << "New Name: "; getline(cin, newName);
141             unifiedDelete(oldPhone);
142             addContact(newName, oldPhone);
143             cout << "Name updated.\n";
144         }
145         else if (upChoice == 2) { // Change Number only
146             string newPhone; cout << "New Number: "; getline(cin, newPhone);
147             strncpy(target->data.phone, newPhone.c_str(), 14);
148             cout << "Number updated.\n";
149         }
150         else if (upChoice == 3) { // Change Both
151             string newName, newPhone;
152             cout << "New Name: "; getline(cin, newName);
153             cout << "New Number: "; getline(cin, newPhone);
154             unifiedDelete(oldPhone);
155             addContact(newName, newPhone);
156             cout << "Both updated.\n";
157         }
158         else { cout << "Invalid selection.\n"; }
159     }
160
```

```
161    void displayAll() {
162        Node* curr = head;
163        if (!curr) { cout << "\nDirectory Empty.\n"; return; }
164        cout << "\n" << left << setw(20) << "NAME" << setw(15) << "PHONE" << endl
               ;
165        cout << "-----------------------------------\n";
166        while (curr) {
167            cout << left << setw(20) << curr->data.name << setw(15) << curr->data
                   .phone << endl;
168            curr = curr->next;
169        }
170    }
171
172    void showStats() {
173        int count = 0; Node* curr = head;
174        while (curr) { count++; curr = curr->next; }
175        cout << "\nTotal Contacts: " << count << endl;
176    }
177
178    void saveToBinary() {
179        ofstream outFile("phonebook.dat", ios::binary | ios::trunc);
180        Node* curr = head;
181        while (curr) {
182            outFile.write((char*)&curr->data, sizeof(ContactData));
183            curr = curr->next;
184        }
185    }
186
```

```
187    void loadFromBinary() {
188        ifstream inFile("phonebook.dat", ios::binary);
189        if (!inFile) return;
190        ContactData temp;
191        while (inFile.read((char*)&temp, sizeof(ContactData))) addContact(temp
               .name, temp.phone);
192    }
193 };
194
195 int main() {
196     Phonebook myBook;
197     int choice;
198     string input, phone;
199
200     do {
201         cout << "\n--- PHONEBOOK DIRECTORY ---";
202         cout << "\n1. Add Contact\n2. Search (Name/No)\n3. Update Contact\n4.
                Delete (Name/No)\n5. View All\n6. Stats\n7. Exit\nChoice: ";
203         if (!(cin >> choice)) { cin.clear(); cin.ignore(1000, '\n'); continue; }
204         cin.ignore();
205
206         switch (choice) {
207             case 1: cout << "Name: "; getline(cin, input); cout << "Phone: ";
                    getline(cin, phone); myBook.addContact(input, phone); break;
208             case 2: cout << "Search Query: "; getline(cin, input); myBook
                    .unifiedSearch(input); break;
209             case 3: cout << "Update Query: "; getline(cin, input); myBook
                    .updateContact(input); break;
```

```
196     Phonebook myBook;
197     int choice;
198     string input, phone;
199
200     do {
201         cout << "\n--- PHONEBOOK DIRECTORY ---";
202         cout << "\n1. Add Contact\n2. Search (Name/No)\n3. Update Contact\n4.
                Delete (Name/No)\n5. View All\n6. Stats\n7. Exit\nChoice: ";
203         if (!(cin >> choice)) { cin.clear(); cin.ignore(1000, '\n'); continue; }
204         cin.ignore();
205
206         switch (choice) {
207             case 1: cout << "Name: "; getline(cin, input); cout << "Phone: ";
                    getline(cin, phone); myBook.addContact(input, phone); break;
208             case 2: cout << "Search Query: "; getline(cin, input); myBook
                    .unifiedSearch(input); break;
209             case 3: cout << "Update Query: "; getline(cin, input); myBook
                    .updateContact(input); break;
210             case 4: cout << "Delete Query: "; getline(cin, input); myBook
                    .unifiedDelete(input); break;
211             case 5: myBook.displayAll(); break;
212             case 6: myBook.showStats(); break;
213             case 7: cout << "Exiting...\n"; break;
214             default: cout << "Invalid Choice.\n";
215         }
216     } while (choice != 7);
217     return 0;
218 }
```

Output

```
--- PHONEBOOK DIRECTORY ---
1. Add Contact
2. Search (Name/No)
3. Update Contact
4. Delete (Name/No)
5. View All
6. Stats
7. Exit
Choice:
=== Session Ended. Please Run the code again ===
```